A project report on

Video-Based Fruit Counting with Tracking to Eliminate Double-Counting

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering Core

by

DERRICK SAMUEL R. (23BCE1040)

PRANNAVAKHANTH (23BCE1869)

PRIYANKA KALIRAJ (23BCE5131)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

A project report on

Video-Based Fruit Counting with Tracking to Eliminate Double-Counting

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering Core

by

DERRICK SAMUEL R. (23BCE1040)

PRANNAVAKHANTH (23BCE1869)

PRIYANKA KALIRAJ (23BCE5131)



SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

November, 2026

DECLARATION

I hereby declare that the thesis entitled “Video-Based Fruit Counting with Tracking to Eliminate Double-Counting” submitted by DERRICK SAMUEL R. (23BCE1040), PRANNAVAKHANTH (23BCE1869), and PRIYANKA KALIRAJ (23BCE5131), for the award of the degree of Bachelor of Technology in Computer Science and Engineering, Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr. Manas Ranjan Prusty.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai **Signature of the Candidate**

Date:

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled **"Video-Based Fruit Counting with Tracking to Eliminate Double-Counting"** is prepared and submitted by DERRICK SAMUEL R. (23BCE1040), PRANNAVAKHANTH (23BCE1869), and PRIYANKA KALIRAJ (23BCE5131) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering Core** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. Manas Ranjan Prusty

Date:

Signature of the Examiner

Name: Dr. Renta Chintala Bhargavi

Date:

Signature of the Examiner

Name: Dr. Ranjith R

Date:

Approved by the Head of Department,

B.Tech. Computer Science and Engineering Core

Name: **Dr. Prassanna Jayachandran**

Date:

ABSTRACT

OrchardVision is a computer vision system for long-term fruit tracking and counting in mango orchards. It implements identity learning, tracking pipelines, and a full-stack web interface for experiment management and video processing.

Estimating the number of fruit on a tree or across a block is central to yield forecasting, harvest logistics and crop-load management, yet it is still largely done by manual sampling. Counting from orchard video is attractive but difficult: fruit are small, visually similar, frequently hidden by leaves and branches, and the same fruit re-enters the camera view many times as the camera moves along a row or revisits it on later days. A detector-only counter therefore double-counts, and a frame-to-frame tracker alone loses identities whenever a fruit is occluded.

The system developed in this project is built around a processing pipeline that orchestrates frame extraction, object detection, tracking (mock, ByteTrack or TAPNext backends), re-identification embeddings, identity consolidation and analytics. Each fruit is represented by an identity prototype (an embedding vector with quality and observation statistics), so that a fruit that disappears and later reappears can be re-associated with the same identity. Camera motion is estimated and compensated, and tracking quality is reported using IDF1, HOTA and identity-switch metrics. The backend is a FastAPI service with a SQLAlchemy 2.0 data layer (SQLite for development, PostgreSQL for production) and S3-compatible object storage; the frontend is a React and TypeScript application sharing typed contracts with the API. An experiment harness lets users run and compare tracker configurations and export results as JSON or CSV.

The report presents the motivation, related work, requirements, architecture, data model, pipeline design, API design, deployment and testing approach of OrchardVision, followed by an evaluation plan with placeholders for the quantitative results and a discussion of limitations and future work.

Keywords: fruit counting; multi-object tracking; re-identification; identity learning; precision agriculture; mango orchard; FastAPI; React.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. Manas Ranjan Prusty, Associate Professor Grade 2, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for his constant guidance, continual encouragement, understanding; more than all, he taught me patience in my endeavor. My association with him is not confined to academics only, but it is a great opportunity on my part of work with an intellectual and expert in the field of computer vision and machine learning.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to **Dr. Prassanna Jayachandran**, Head of the Department, B.Tech. **Computer Science and Engineering (Core)** and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai **DERRICK SAMUEL R.**

Date:

CONTENTS

LIST OF ACRONYMS viii

LIST OF SYMBOLS AND NOMENCLATURE ix

CHAPTER 1 Introduction 1
- 1.1 Introduction . . . 1
- 1.2 Overview of Orchard Fruit Counting . . . 1
- 1.3 Challenges in Orchard Video . . . 2
- 1.4 Motivation . . . 2
- 1.5 Project Statement . . . 3
- 1.6 Objectives . . . 3
- 1.7 Scope of the Project . . . 4
- 1.8 Organisation of the Report . . . 4

CHAPTER 2 Background and Literature Survey 5
- 2.1 Introduction . . . 5
- 2.2 Fruit Detection in Orchards . . . 5
- 2.3 Tracking-by-Detection . . . 5
  - 2.3.1 SORT and DeepSORT . . . 5
  - 2.3.2 ByteTrack . . . 5
  - 2.3.3 BoT-SORT and Camera Motion Compensation . . . 6
- 2.4 Re-Identification and Identity Learning . . . 6
- 2.5 Tracking Any Point . . . 6
- 2.6 Evaluation Metrics for Tracking . . . 6
- 2.7 Fruit Counting and Yield Estimation from Video . . . 7
- 2.8 Datasets and Benchmarks . . . 7
- 2.9 Learning Objectives for Embeddings . . . 8
- 2.10 Summary of Related Work . . . 8
- 2.11 Comparison of Tracker Families . . . 8
- 2.12 Research Gap . . . 8

CHAPTER 3 System Requirements and Architecture 10
- 3.1 Introduction . . . 10
- 3.2 Requirements . . . 10
  - 3.2.1 Functional Requirements . . . 10
  - 3.2.2 Non-Functional Requirements . . . 10
- 3.3 High-Level Architecture . . . 11
- 3.4 Backend . . . 11
- 3.5 Frontend . . . 12

3.6 Core Services . . . 12
3.7 Configuration . . . 12
3.8 Actors and Use Cases . . . 12
3.9 Hardware and Software Requirements . . . 14
3.10 Technology Stack . . . 14
3.11 Repository Organisation . . . 14

CHAPTER 4 Data Model and Persistence 15
4.1 Introduction . . . 15
4.2 Entities . . . 15
4.3 Entity Relationships . . . 15
4.4 Rows Created by One Processing Run . . . 16
4.5 Identity Records . . . 16
4.6 Tracks and Detections . . . 17
4.7 Telemetry and Camera Motion . . . 17
4.8 Settings and Experiments . . . 17
4.9 Database Selection . . . 17

CHAPTER 5 Processing Pipeline and Identity Learning 18
5.1 Introduction . . . 18
5.2 End-to-End Flow . . . 18
5.2.1 Upload . . . 18
5.2.2 Process . . . 18
5.2.3 Retrieve . . . 19
5.3 Status Models . . . 19
5.4 Processing Stages . . . 19
5.4.1 Frame Extraction . . . 19
5.4.2 Object Detection . . . 20
5.4.3 Tracking . . . 20
5.4.4 Re-Identification Embeddings . . . 20
5.4.5 Identity Consolidation . . . 20
5.4.6 Analytics . . . 21
5.5 Stage Inputs and Outputs . . . 21
5.6 Camera Motion . . . 21
5.7 Identity-Space Visualisation . . . 21
5.8 Identity Consolidation Procedure . . . 22
5.9 Design Considerations . . . 22
5.10 Computational Considerations . . . 23
5.11 Pipeline Parameters . . . 23
5.12 Failure Handling . . . 23
5.13 Reproducibility . . . 23
5.14 Concurrency and Job Control . . . 24

CHAPTER 6 API Design and Implementation 25
6.1 Introduction . . . 25
6.2 Endpoint Summary . . . 25
6.3 Resource-to-Model Mapping . . . 26
6.4 Request Flows . . . 26
6.4.1 Video Upload . . . 26

6.4.2 Starting a Processing Job . . . 27
6.4.3 Reading Results . . . 27
6.4.4 Experiments . . . 27
6.4.5 Settings . . . 27
6.5 Error Handling . . . 28
6.6 Shared TypeScript Types . . . 28
6.7 Frontend Integration . . . 28
6.8 Example Interaction . . . 28

CHAPTER 7 Deployment, Testing and Extension 30
7.1 Introduction . . . 30
7.2 Development Environment . . . 30
7.3 Production Deployment . . . 30
7.4 Configuration . . . 30
7.5 Extending the System . . . 31
7.6 Testing . . . 31
7.7 Security and Robustness Considerations . . . 31
7.8 User Guide . . . 32
7.9 Project Plan . . . 33
7.10 Risk Analysis . . . 33

CHAPTER 8 Experimental Evaluation and Results 34
8.1 Introduction . . . 34
8.2 Experiment Harness . . . 34
8.3 Experimental Set-up . . . 34
8.3.1 Data . . . 34
8.3.2 Configurations . . . 34
8.3.3 Metrics . . . 35
8.4 Tracker Comparison . . . 35
8.5 Effect of Camera Motion Compensation . . . 36
8.6 Identity Prototypes and Re-Identification . . . 36
8.7 Sensitivity to the ReID Threshold . . . 36
8.8 Counting Accuracy . . . 38
8.9 Runtime Performance . . . 38
8.10 Web Interface . . . 38
8.11 Discussion . . . 40

CHAPTER 9 Conclusion and Future Work 41
9.1 Conclusion . . . 41
9.2 Future Work . . . 41

REFERENCES 43

APPENDIX 1: REPOSITORY LAYOUT 45

APPENDIX 2: RUN COMMANDS 46

APPENDIX 3: ILLUSTRATIVE API RESPONSES 47

LIST OF FIGURES

3.1 High-level architecture of OrchardVision 11
3.2 Use-case diagram 13

4.1 Overview of the data model 16

5.1 Processing pipeline from upload to persisted results. 19
5.2 Life cycle of a `ProcessingJob` (`ProcessingStatus`). 19

6.1 Sequence diagram of a processing request 27

7.1 Production deployment topology. 30
7.2 User workflow 32

8.1 Sample orchard frames 35
8.2 Tracker comparison chart 35
8.3 Identity embedding projection 36
8.4 Re-identification example 36
8.5 Threshold sweep 37
8.6 Per-frame processing time 38
8.7 Video list and upload screen 38
8.8 Track visualisation 39
8.9 Experiment management screen 39
8.10 Settings screen 39

LIST OF TABLES

2.1 Summary of related work 8
2.2 Comparison of tracker families 9

3.1 Hardware and software requirements 13
3.2 Technology stack of OrchardVision. 13

4.1 Data models and key fields. 15
4.2 Rows per processed video 16

5.1 Stage inputs and outputs 21
5.2 Key pipeline parameters (defaults stored in `AppSettings`). 23

6.1 REST API endpoints. 25
6.2 Endpoint-to-model mapping 26

7.1 Recommended test plan 31
7.2 Project plan 33
7.3 Risk analysis 33

8.1 Evaluation videos 34
8.2 Tracker comparison 35
8.3 Camera motion ablation 36
8.4 ReID threshold sensitivity 37
8.5 Counting accuracy 37
8.6 Processing time 38

LIST OF ACRONYMS

API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
FPS	Frames Per Second
HOTA	Higher Order Tracking Accuracy
IDF1	Identification F1 score
IDSW	Identity Switches (ID switches)
JSON	JavaScript Object Notation
MOT	Multi-Object Tracking
ORM	Object-Relational Mapping
PCA	Principal Component Analysis
ReID	Re-Identification
REST	Representational State Transfer
S3	Simple Storage Service (object-storage API)
SQL	Structured Query Language
TAP	Tracking Any Point
t-SNE	t-distributed Stochastic Neighbour Embedding
UI	User Interface
UUID	Universally Unique Identifier

LIST OF SYMBOLS AND NOMENCLATURE

$\mathbf{e}_t$	ReID embedding extracted from a detection at frame t
$\mathbf{p}_k$	Prototype (memory) embedding of identity k
α	Memory update rate used when a prototype is refreshed
τ_{reid}	ReID similarity threshold for accepting a re-association
τ_{conf}	Detection confidence threshold
τ_{match}	Tracker match threshold
IDTP, IDFP, IDFN	Identity-level true positives, false positives and false negatives
HOTA_β	HOTA score at localisation threshold β
DetA, AssA	Detection accuracy and association accuracy components of HOTA

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

OrchardVision is a computer vision system for long-term fruit tracking and counting in mango orchards. It implements identity learning, tracking pipelines, and a full-stack web interface for experiment management and video processing. A user uploads orchard video through the web interface; the backend extracts frames, detects fruit, tracks each fruit through time, learns a re-identification (ReID) embedding for it, consolidates the observations into persistent identities and finally computes analytics such as IDF1, HOTA and the number of identity switches.

Mango is one of the most widely grown tropical fruit crops, and the timing of harvest, labour planning, packing-house capacity and market commitments all depend on how many fruit an orchard will deliver. Growers traditionally estimate this by hand-counting a small sample of trees and extrapolating to the whole block. The process is slow, subjective and statistically weak, because fruit load varies strongly from tree to tree and from row to row.

Video captured by a handheld camera, a tractor-mounted camera or an unmanned aerial vehicle offers a much denser sample of the orchard. Turning that video into a reliable count, however, is not simply a matter of running an object detector on every frame. Each fruit is visible in dozens or hundreds of consecutive frames, and a detector will report it every time. The central problem is therefore one of *identity*: deciding whether a detection in the current frame is a fruit that has already been counted or a new one.

1.2 OVERVIEW OF ORCHARD FRUIT COUNTING

Automatic fruit counting systems typically combine three ingredients. A detector finds fruit in each frame; a tracker links detections of the same fruit across consecutive frames into short trajectories; and a counting rule converts trajectories into a number, for example by counting the number of unique trajectories or by counting crossings of a virtual line. This design works acceptably when fruit are always visible and the camera moves smoothly in one direction.

Real orchards violate both assumptions. Foliage and branches hide fruit for tens of frames at a time, which breaks trajectories and inflates the count when the fruit reappears.

In long-term monitoring the same tree is recorded on several days, or the camera returns to a row after a turn, so identities must survive gaps of seconds, minutes or days. OrchardVision addresses this by adding an *identity learning* layer on top of tracking: every identity is represented by a prototype embedding that is updated as new observations arrive, and tracks that end and later reappear can be linked back to the same identity through a ReID step.

The system is organised as a pipeline (frame extraction, detection, tracking, ReID embedding, identity consolidation, analytics), a REST API that exposes every intermediate and final result, a relational data model that persists them, and a web client with an experiment harness for comparing configurations.

1.3 CHALLENGES IN ORCHARD VIDEO

The following factors make fruit tracking in orchards considerably harder than the pedestrian and vehicle tracking problems on which most multi-object tracking research has been developed:

- **Occlusion.** Leaves, branches and other fruit frequently hide a fruit partially or completely, so tracks fragment.
- **Similar appearance.** Fruit on the same tree share colour, shape and texture, which weakens appearance cues that ReID relies on.
- **Camera motion.** Handheld or vehicle-mounted cameras move constantly, so image-space motion models (such as constant-velocity Kalman filters) are disturbed by ego-motion rather than by object motion.
- **Illumination and scale changes.** Sunlight, shadows and varying distance to the canopy change the apparent size and colour of fruit.
- **Long gaps.** In long-term monitoring an identity may disappear for far longer than any tracker buffer, so matching must rely on learned appearance rather than short-term motion.
- **Dense small objects.** A single frame may contain tens of small, closely spaced fruit, increasing the risk of identity switches.

1.4 MOTIVATION

Three observations motivate the project. First, yield estimates are needed early and repeatedly during the season, and manual sampling cannot keep up with this demand at the scale of commercial orchards. Second, video is cheap to collect and is already captured during routine orchard operations, so a counting method that runs on ordinary

video is easy to adopt. Third, counting errors from video are dominated by identity errors rather than detection errors, which means that progress requires explicit support for identity: storing what each fruit looks like, remembering it after it disappears and measuring how well this works.

A further motivation is engineering. Research prototypes in this area are often a script that prints a number. A reusable system needs persistent results, a stable API, reproducible experiments and a user interface, so that agronomists and researchers can examine and trust the output. OrchardVision therefore treats the data model, the API and the experiment harness as first-class parts of the contribution.

1.5 PROJECT STATEMENT

To design and implement an end-to-end, web-based system that ingests orchard video, detects and tracks individual mango fruit, maintains a persistent learned identity for each fruit across occlusions and re-appearances, and reports tracking quality with standard metrics, while making every stage configurable and every result inspectable and comparable through a reproducible experiment harness.

1.6 OBJECTIVES

The objectives of the project are:

1. To implement a processing pipeline that orchestrates frame extraction, detection, tracking, ReID embedding, identity consolidation and analytics for an uploaded video.
2. To support multiple tracker back-ends (mock, ByteTrack and TAPNext) behind a common interface so that they can be compared on the same video.
3. To learn and persist an identity prototype (embedding vector, quality and observation count) for every fruit and to record ReID events when an identity is lost and later reappears.
4. To estimate camera motion and record whether compensation was applied.
5. To compute IDF1, HOTA and ID-switch metrics and per-frame timing telemetry.
6. To expose all results through a documented REST API and a React / TypeScript web client with shared type definitions.
7. To provide an experiment harness that runs and stores tracker evaluations and exports them as JSON or CSV.
8. To make the system deployable on a developer laptop (SQLite) and in production (PostgreSQL, object storage, background workers).

1.7 SCOPE OF THE PROJECT

The scope of the work covers the backend service, the data model, the processing pipeline, the REST API, the shared TypeScript types, the deployment configuration and the experiment-management interface described in the following chapters. Uploads are limited to 2 GB per video by default (configurable through the `MAX_UPLOAD_BYTES` environment variable).

The following are explicitly outside the scope of this report: training new fruit detectors from scratch, fruit-size or ripeness estimation, three-dimensional orchard mapping, and multi-camera fusion. The evaluation reported in Chapter 8 is limited to the videos and tracker configurations listed there.

1.8 ORGANISATION OF THE REPORT

Chapter 2 reviews the literature on fruit detection, multi-object tracking, re-identification and tracking metrics. Chapter 3 presents the requirements and the system architecture. Chapter 4 describes the data model. Chapter 5 explains the processing pipeline and the identity-learning design. Chapter 6 documents the REST API, error handling and shared types. Chapter 7 covers deployment and testing. Chapter 8 presents the evaluation set-up and results, and Chapter 9 concludes the report and outlines future work. References and appendices follow.

CHAPTER 2

BACKGROUND AND LITERATURE SURVEY

2.1 INTRODUCTION

This chapter reviews the techniques on which OrchardVision builds: deep-learning-based fruit detection, tracking-by-detection, appearance-based re-identification, point tracking, camera-motion compensation and the metrics used to judge tracking quality. The chapter closes with a comparison table and the research gap that motivates the project.

2.2 FRUIT DETECTION IN ORCHARDS

Deep convolutional detectors have replaced hand-crafted colour and shape features for fruit detection. Bargoti and Underwood (2017) showed that a Faster R-CNN detector trained on a few hundred annotated images could detect mangoes, apples and almonds in orchard imagery with high F1 scores, and Koirala et al. (2019) surveyed deep-learning methods for fruit detection and yield estimation, concluding that single-stage detectors from the YOLO family (Redmon et al., 2016) give the best trade-off between accuracy and speed for field deployment. In OrchardVision the detector is a pluggable component: the model is selected through a `model_path` setting, and detections are filtered by a configurable confidence threshold.

2.3 TRACKING-BY-DETECTION

2.3.1 SORT AND DEEPSORT

SORT (Bewley et al., 2016) links detections across frames with a Kalman filter (Kalman, 1960) for motion prediction and the Hungarian algorithm (Kuhn, 1955) for assignment on an intersection-over-union cost. It is fast but suffers many identity switches under occlusion. DeepSORT (Wojke et al., 2017) adds a learned appearance embedding to the association cost, which substantially reduces identity switches at the price of an extra network evaluation per detection.

2.3.2 BYTETRACK

ByteTrack (Zhang et al., 2022) observes that low-confidence detections are often occluded objects rather than noise. It associates high-confidence detections first and then

uses the low-confidence ones to recover tracks that would otherwise be lost. This is particularly relevant for fruit, where partial occlusion lowers detector confidence. ByteTrack is one of the tracker back-ends supported by OrchardVision.

2.3.3 BOT-SORT AND CAMERA MOTION COMPENSATION

BoT-SORT (Aharon et al., 2022) combines motion and appearance information and introduces camera-motion compensation, in which the global image motion between frames is estimated and used to correct the Kalman-filter predictions. Because orchard video is almost always recorded from a moving platform, compensating for camera motion is expected to matter more than in fixed-camera surveillance. OrchardVision records the estimated motion type and whether compensation was applied for every processed video.

2.4 RE-IDENTIFICATION AND IDENTITY LEARNING

Re-identification originates in person re-identification, where a network maps an image crop to an embedding such that crops of the same identity are close and crops of different identities are far apart. Hermans et al. (2017) showed that a carefully mined triplet loss is a strong baseline for this task. For tracking, an identity can be summarised by a *prototype* embedding obtained by aggregating the embeddings of its observations; new detections are compared with prototypes, and a similarity above a threshold triggers re-association. This is the approach followed by OrchardVision: each identity stores a prototype vector, a quality score, an observation count and a flag indicating whether the identity is confident.

2.5 TRACKING ANY POINT

Point-tracking methods follow an arbitrary query point through a video, even through occlusion. TAP-Vid (Doersch et al., 2022) introduced a benchmark for this task, and TAPNext (Zholus et al., 2025) formulates point tracking as next-token prediction in a sequence model. Because fruit are small and lack distinctive texture, tracking a point on the fruit surface can be more stable under occlusion than tracking a bounding box. TAPNext is available in OrchardVision as one of the tracker options.

2.6 EVALUATION METRICS FOR TRACKING

Three families of metrics are used in this work. The CLEAR MOT metrics (Bernardin and Stiefelhagen, 2008) count identity switches (IDSW), false positives and misses.

IDF1 (Ristani et al., 2016) measures how long identities are correctly maintained and is defined as

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}} \tag{2.1}$$

where IDTP, IDFP and IDFN are identity-level true positives, false positives and false negatives obtained from a global one-to-one matching between ground-truth and predicted trajectories. HOTA (Luiten et al., 2021) balances detection and association quality:

$$\mathrm{HOTA}_\beta = \sqrt{\mathrm{DetA}_\beta \cdot \mathrm{AssA}_\beta} \tag{2.2}$$

and is usually reported as the average over a range of localisation thresholds β. For fruit counting, IDF1 and the number of identity switches are especially informative, because every identity switch changes the count.

2.7 FRUIT COUNTING AND YIELD ESTIMATION FROM VIDEO

Liu et al. (2019) proposed monocular-camera fruit counting with semantic data association, where fruit are tracked across frames and associated with a 3-D map to avoid double counting. Their work demonstrates the value of explicit data association but relies on structure-from-motion, which is expensive and brittle in foliage-rich scenes. OrchardVision pursues a complementary, appearance-based route: identities are maintained by learned embeddings and can be evaluated through the same experiment harness as any other tracker.

2.8 DATASETS AND BENCHMARKS

Tracking research is usually benchmarked on pedestrian and vehicle datasets, in which the metrics above were defined, whereas orchard datasets for tracking are scarce. Point-tracking has its own benchmark in TAP-Vid (Doersch et al., 2022). Because public orchard-tracking benchmarks with persistent fruit identities are limited, a practical evaluation of OrchardVision requires manually annotated orchard videos in which each fruit is assigned a persistent identifier. The experiment harness of the system is designed to run the same configurations on any such video and to export the metrics for comparison.

2.9 LEARNING OBJECTIVES FOR EMBEDDINGS

Embedding networks for re-identification are commonly trained with classification losses, with metric-learning losses such as the triplet loss (Hermans et al., 2017), or with a combination of both. For fruit, the main difficulty is that different instances look alike, so metric-learning losses with hard-example mining are attractive: they explicitly push embeddings of different fruit apart and pull embeddings of the same fruit together across viewpoints and lighting. The quality of the embedding determines how reliably identity matching can bridge long gaps, which is why the identity prototype in OrchardVision stores a quality score and an observation count along with the embedding.

2.10 SUMMARY OF RELATED WORK

Work	Technique	Strength	Limitation
Bargoti & Underwood (2017)	Faster R-CNN fruit detection	Accurate per-frame detection	No temporal identity
Bewley et al. (2016)	SORT: Kalman + Hungarian	Very fast	Many ID switches under occlusion
Wojke et al. (2017)	DeepSORT: adds appearance embedding	Fewer ID switches	Extra embedding cost; limited long gaps
Zhang et al. (2022)	ByteTrack: uses low-score boxes	Recovers occluded objects	Motion-based; no learned identity memory
Aharon et al. (2022)	BoT-SORT: motion + appearance + camera compensation	Robust to ego-motion	Designed for pedestrians
Liu et al. (2019)	Fruit counting with semantic data association	Avoids double counting through 3-D map	Requires structure from motion
Zholus et al. (2025)	TAPNext point tracking	Tracks through occlusion	Point-based; needs query points

Table 2.1: Summary of related work on detection, tracking and fruit counting.

2.11 COMPARISON OF TRACKER FAMILIES

2.12 RESEARCH GAP

Existing fruit-counting work either focuses on single-frame detection or on short-term tracking, and rarely exposes the intermediate artefacts (tracks, identity prototypes, ReID events, per-frame timing, camera motion) needed to understand *why* a count is wrong. There is also little tooling for repeatable comparison of tracker configurations on the

Family	Association cue	Handles long occlusion	Typical cost
Motion only (SORT, ByteTrack)	Kalman prediction and IoU	Poorly beyond the track buffer	Low
Motion + appearance (DeepSORT, BoT-SORT)	Embedding distance combined with motion	Moderately	Medium (embedding per detection)
Point tracking (TAPNext)	Point trajectories through the video	Well, if the point remains trackable	Higher (sequence model)
Identity memory (this work)	Prototype embeddings with ReID threshold	Designed for long gaps	Medium (embedding and matching)

Table 2.2: Qualitative comparison of tracker families.

same orchard video. OrchardVision addresses both gaps by combining identity learning with a persistent, queryable data model and an experiment harness.

CHAPTER 3

SYSTEM REQUIREMENTS AND ARCHITECTURE

3.1 INTRODUCTION

OrchardVision is a full-stack system consisting of a FastAPI backend, a React + TypeScript frontend and a set of core services that implement the computer vision pipeline. This chapter states the requirements that the system satisfies and describes its architecture and technology stack.

3.2 REQUIREMENTS

3.2.1 FUNCTIONAL REQUIREMENTS

The functional requirements below were derived from the capabilities of the implemented system:

- FR1: upload a video, store it, and extract its metadata (width, height, frame rate and frame count).
- FR2: start, monitor and complete a processing job for an uploaded video, and reject a new job while another is active.
- FR3: persist tracks, detections, identities, ReID events, camera-motion estimates and per-frame timing for each processed video.
- FR4: retrieve every persisted artefact through the REST API, including a two-dimensional projection of identity prototypes.
- FR5: create, run, list, export and delete evaluation experiments for different trackers and scenarios.
- FR6: read and update the default pipeline settings.
- FR7: delete a video together with all associated data.

3.2.2 NON-FUNCTIONAL REQUIREMENTS

- **Portability.** Run with SQLite on a laptop and with PostgreSQL in production without code changes.
- **Responsiveness.** Long-running processing must not block API requests; it is executed as a background task or through a Celery queue.

- **Consistency.** All errors use a single JSON structure and the frontend and backend share type definitions.
- **Extensibility.** New models, routers, schemas and services can be added in separate modules without modifying existing ones.
- **Scalability of storage.** Videos up to 2 GB are stored in S3-compatible object storage rather than in the database.

3.3 HIGH-LEVEL ARCHITECTURE

Figure 3.1 shows the components of OrchardVision and their interaction. The React client talks to the FastAPI backend over REST under the `/api/v1/` prefix; the backend uses the core services to process videos, persists results through SQLAlchemy and stores binary data in MinIO/S3.

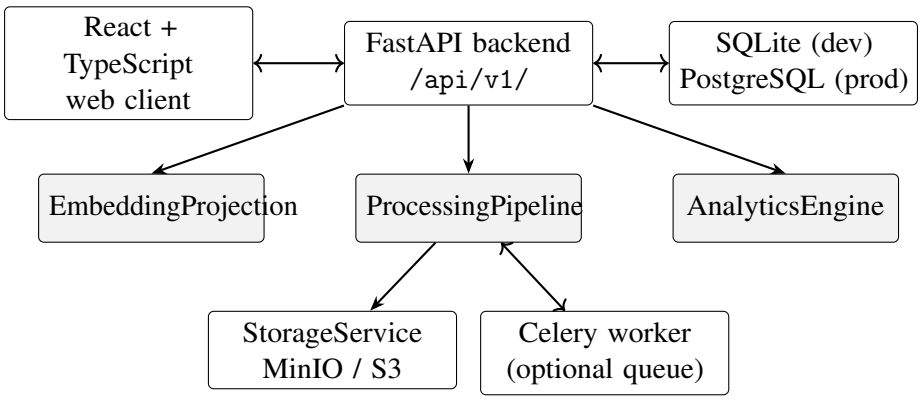


Figure 3.1: High-level architecture of OrchardVision showing the web client, FastAPI backend, core services, relational database, object storage and optional Celery worker.

3.4 BACKEND

The backend is implemented with the following choices:

- **Framework:** FastAPI with the SQLAlchemy 2.0 ORM.
- **Database:** SQLite for development and PostgreSQL for production, selected through the `DATABASE_URL` environment variable.
- **API:** REST under `/api/v1/`, organised in router prefixes for health (`GET /health`), videos (`POST/GET/DELETE /videos`), tracks (`GET /videos/{id}/tracks`), identities (`GET /videos/{id}/identities`), processing (`POST /videos/{id}/process`), jobs (`GET /jobs/{id}`), experiments (`GET/POST/DELETE /experiments`) and settings (`GET/PUT /settings`).

3.5 FRONTEND

The frontend is a React and TypeScript application. Types shared with the API are defined once in `packages/types/src/`, the components live in `apps/web/src/`, and an API client calls the FastAPI backend. During development the Vite server proxies requests under `/api/` to the backend.

3.6 CORE SERVICES

- **ProcessingPipeline.** Orchestrates frame extraction → detection → tracking → embedding → analytics.
- **EmbeddingProjection.** Produces t-SNE / PCA visualisations of identity prototypes, served as two-dimensional coordinates.
- **AnalyticsEngine.** Computes IDF1, HOTA and ID-switch metrics.
- **StorageService.** Provides MinIO/S3-backed file management for uploads and frames.

3.7 CONFIGURATION

Configuration is loaded through `pydantic_settings` from an `.env` file. The key settings are the confidence threshold, the ReID threshold, the tracker type, the camera-motion mode, the memory update rate, the tracker match threshold and the number of buffer frames. The defaults are persisted in the `AppSettings` table, which holds a single row identified by `GLOBAL_SETTINGS_ID`, and can be read and updated through `GET /settings` and `PUT /settings`.

3.8 ACTORS AND USE CASES

Two kinds of users interact with the system. An *analyst* (for example an agronomist or a student researcher) uploads videos, starts processing, inspects tracks and identities and reads analytics. An *experimenter* additionally creates and compares experiments and edits the pipeline defaults. The main use cases are: upload video, process video, view tracks, view identities and their projection, view ReID events, view analytics, manage experiments, export results, edit settings and delete video.

[IMAGE PLACEHOLDER]

UML use-case diagram. Two actors on the left (Analyst, Experimenter) and the system boundary "OrchardVision" containing the ten use cases listed above. Draw Experimenter as a specialisation of Analyst, and connect "Process video" with an `include` relationship to "Upload video".

Figure 3.2: Use-case diagram of OrchardVision.

Item	Requirement
Operating system	Linux, macOS or Windows capable of running Python, Node.js and Docker
Python	Python environment with FastAPI, SQLAlchemy 2.0, pydantic-settings, uvicorn/gunicorn, Celery and the detection/tracking libraries used by the pipeline
Node.js	Node.js with npm and pnpm for the frontend and the shared types package
Database	SQLite (development) or PostgreSQL (production)
Object storage	MinIO or any S3-compatible service
Compute	<<CPU / GPU model and memory used for the experiments>>
Browser	A current desktop browser (the frontend is served by Vite in development)

Table 3.1: Hardware and software requirements.

Layer	Technology
Web framework	FastAPI served by uvicorn (gunicorn/uvicorn in production)
ORM / database	SQLAlchemy 2.0; SQLite (development), PostgreSQL (production)
Settings	`pydantic_settings` loading `.env`
Background work	FastAPI background task, or Celery when a queue is configured
Object storage	MinIO / S3
Frontend	React, TypeScript, Vite
Shared contracts	TypeScript package in `packages/types/src/`
Testing / checks	`pytest`; `pnpm exec tsc --noEmit`

Table 3.2: Technology stack of OrchardVision.

3.9 HARDWARE AND SOFTWARE REQUIREMENTS

3.10 TECHNOLOGY STACK

3.11 REPOSITORY ORGANISATION

New models and migrations are added as SQLAlchemy 2.0 model definitions. New API routes live in `apps/api/app/routers/`, schemas in `apps/api/app/schemas/` and services in `apps/api/app/services/`. The application entry point and the global exception handler are in `apps/api/app/main.py`. The complete layout is given in Appendix A.

CHAPTER 4

DATA MODEL AND PERSISTENCE

4.1 INTRODUCTION

All results produced by the pipeline are persisted so that they can be retrieved later through the API without reprocessing the video. The data layer is defined with SQLAlchemy 2.0 model definitions. Most tables carry a `video_id` that links them to the `Video` row they were derived from, which is why deleting a video (`DELETE /videos/{id}`) removes all associated data.

4.2 ENTITIES

Table 4.1 lists the persisted models and their key fields.

Table 4.1: Data models and key fields.

Model	Key fields
`Video`	`id`, `filename`, `width`, `height`, `fps`, `frame_count`, `status`
`ProcessingJob`	`id`, `video_id`, `status`, `stage`, `progress`, `config` (dict)
`Experiment`	`id`, `name`, `tracker`, `scenario`, `num_frames`, `status`, `metrics`
`IdentityRecord`	`id`, `video_id`, `identity_id`, `prototype` (embedding vector), `quality`, `observation_count`, `is_confident`
`Track`	`id`, `video_id`, `track_id`, `frames`, `first_seen`, `last_seen`
`DetectionRecord`	`id`, `video_id`, `frame_id`, `bbox`, `confidence`
`FrameTimingRecord`	`id`, `video_id`, `frame_id`, `proc_ms`
`CameraMotionRecord`	`id`, `video_id`, `motion_type`, `compensation_applied`
`IdentityObservation`	`id`, `video_id`, `identity_id`, `track_id`, `frame_id`
`ReidEvent`	`id`, `video_id`, `lost_frame_id`, `reappearance_frame_id`
`AppSettings`	`id` (=`GLOBAL_SETTINGS_ID`), `values` (dict)

4.3 ENTITY RELATIONSHIPS

Figure 4.1 gives an overview of how the models relate to each other. The relationships shown are inferred from the `video_id`, `identity_id`, `track_id` and `frame_id` fields listed above; `Experiment` and `AppSettings` are independent of any single video.

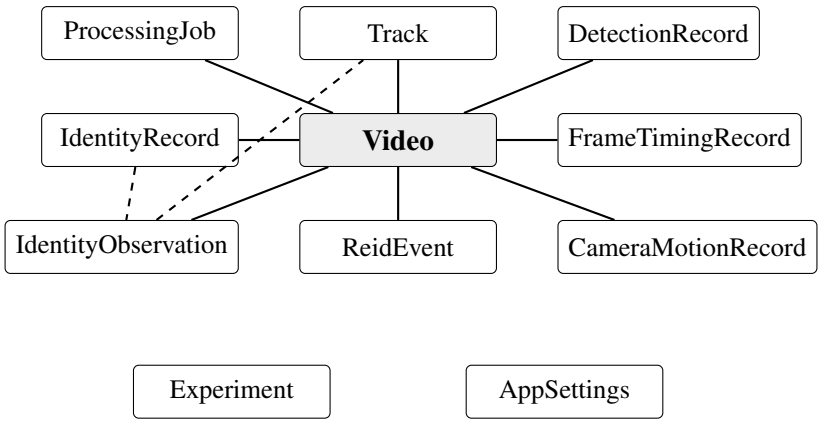


Figure 4.1: Overview of the data model: all per-video artefacts reference `Video`; `Experiment` and `AppSettings` are standalone.

4.4 ROWS CREATED BY ONE PROCESSING RUN

When a processing job completes, the `Video` status becomes `processed` and the following rows are created: `IdentityRecord`, `Track`, `DetectionRecord`, `FrameTimingRecord` and `CameraMotionRecord`; identity sightings and re-identification events are stored as `IdentityObservation` and `ReidEvent` rows. Table 4.2 gives the template used to report the volume of data produced for a video.

Model	Rows per video	Growth driver
`DetectionRecord`	<<XX>>	Frames $\times$ detections per frame
`FrameTimingRecord`	<<XX>>	One per processed frame
`Track`	<<XX>>	Number of tracks started
`IdentityRecord`	<<XX>>	Number of distinct fruit
`IdentityObservation`	<<XX>>	Sightings of identities
`ReidEvent`	<<XX>>	Occlusion/re-appearance events
`CameraMotionRecord`	<<XX>>	Motion estimate(s) for the video

Table 4.2: Rows created per processed video (counts to be filled in from a real run; the growth drivers are inferred).

4.5 IDENTITY RECORDS

An `IdentityRecord` stores the learned prototype of one fruit: an embedding vector, a quality score, the number of observations that contributed to it, and an `is_confident` flag. Each individual sighting of an identity in a frame, attributed to a track, is stored as an `IdentityObservation`. A `ReidEvent` records the frame in which an identity was lost and the frame in which it reappeared.

This split keeps the compact summary (the prototype) separate from the evidence that produced it (the observations), which allows the interface to show both the identity-space visualisation and the per-frame history of any identity.

4.6 TRACKS AND DETECTIONS

A `Track` holds the frames over which a tracker followed an object, together with its first and last frame. `DetectionRecord` stores the bounding box and confidence of each persisted detection. In the shared TypeScript types the bounding box is a `BoundingBox` with `x`, `y`, `width` and `height`, and a track point can be in one of three visibility states: `visible`, `occluded` or `lost`.

4.7 TELEMETRY AND CAMERA MOTION

`FrameTimingRecord` stores the processing time per frame in milliseconds (`proc_ms`) and is returned by `GET /videos/{id}/per-frame`. `CameraMotionRecord` stores the type of motion estimated for the video and whether compensation was applied, and is returned by `GET /videos/{id}/motion`.

4.8 SETTINGS AND EXPERIMENTS

`AppSettings` is a single-row table whose `values` dictionary holds the pipeline defaults. `Experiment` stores the name, tracker, scenario, number of frames, status and computed metrics of an evaluation run. An experiment that is in the `RUNNING` state cannot be deleted.

4.9 DATABASE SELECTION

SQLite is used in development, with the database file at `./data/app.db`. In production the `DATABASE_URL` environment variable selects PostgreSQL. Because both are accessed through the SQLAlchemy ORM, the application code is identical; only the connection string changes. Long-running analytical queries and concurrent writers from several workers are the main reasons to prefer PostgreSQL in production.

CHAPTER 5

PROCESSING PIPELINE AND IDENTITY LEARNING

5.1 INTRODUCTION

This chapter explains how an uploaded video becomes a set of tracks, identities and analytics. It first describes the end-to-end flow (upload, process, retrieve), then each processing stage, the state models that describe progress, and the configuration parameters that control behaviour.

5.2 END-TO-END FLOW

5.2.1 UPLOAD

The user posts a video to `POST /videos`. The backend saves the file through `storage_service.save_upload`, extracts its metadata with `extract_video_metadata`, and creates a `Video` row with status `uploaded`.

5.2.2 PROCESS

A request to `POST /videos/{id}/process` first validates that there is no active job for the video. It then creates a `ProcessingJob` with status `QUEUED`, and either enqueues the work (Celery) or runs it synchronously as a background task. The background task performs:

1. frame extraction according to the `PipelineConfig`;
2. object detection, using the model specified by `model_path`;
3. tracking, using the `mock`, `bytetrack` or `tapnext` tracker;
4. re-identification embeddings;
5. identity consolidation;
6. analytics computation (`compute_video_analytics`).

When processing finishes, the `Video` status becomes `processed`, and the `IdentityRecord`, `Track`, `DetectionRecord`, `FrameTimingRecord` and `CameraMotionRecord` rows are created.

5.2.3 RETRIEVE

The GET endpoints return the persisted tracks, identities, analytics, frame timings, motion estimates, ReID events and detections. Retrieval never triggers recomputation.

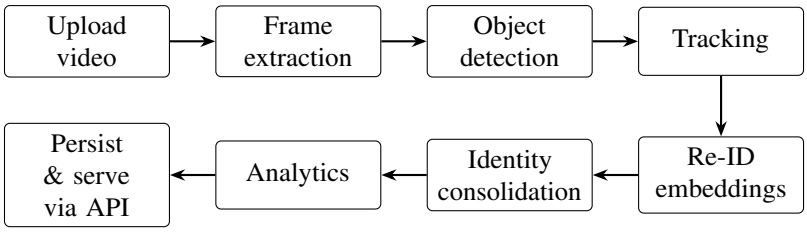


Figure 5.1: Processing pipeline from upload to persisted results.

5.3 STATUS MODELS

The progress of a video and of a processing job is described by the shared TypeScript types in `packages/types/src/common.ts`:

- `JobStatus`: `uploaded` → `processing` → `processed`, or `error`;
- `ProcessingStage`: `frame_extraction` → `detection` → `tracking` → `reid` → `analytics`;
- `ProcessingStatus`: `QUEUED` → `RUNNING` → `COMPLETED`, or `FAILED`.

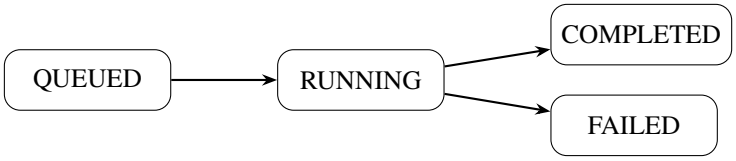


Figure 5.2: Life cycle of a `ProcessingJob` (`ProcessingStatus`).

5.4 PROCESSING STAGES

5.4.1 FRAME EXTRACTION

Frames are extracted from the uploaded video according to the `PipelineConfig` of the job. The extracted frames are stored through the `StorageService` in MinIO/S3, where the video's width, height, frame rate and frame count were recorded at upload time. Sub-sampling the frame rate reduces cost on long videos but increases the displacement of fruit between sampled frames, so it directly affects tracker difficulty.

5.4.2 OBJECT DETECTION

Fruit are detected in each frame by the model specified by `model_path`. Detections below the configured confidence threshold are discarded, and the remainder are persisted as `DetectionRecord` rows with their bounding box and confidence.

5.4.3 TRACKING

The tracker links detections across frames into tracks. The tracker type is selected from `mock`, `bytetrack` and `tapnext`; the tracker match threshold and the number of buffer frames (how long an unmatched track is kept alive) are configurable. The `mock` tracker provides a deterministic back-end for development and tests, ByteTrack is a motion-based tracking-by-detection baseline, and TAPNext follows points through the video (see Chapter 2).

5.4.4 RE-IDENTIFICATION EMBEDDINGS

For each tracked detection the pipeline computes a ReID embedding. The embeddings are used to compare observations with the prototypes of existing identities, using the ReID threshold to decide whether a match is acceptable.

5.4.5 IDENTITY CONSOLIDATION

Identity consolidation turns tracks and embeddings into persistent identities. For each identity the pipeline stores a prototype embedding, a quality score, the observation count and an `is_confident` flag; every sighting is recorded as an `IdentityObservation`. When an identity is lost and later reappears, a `ReidEvent` stores the lost frame and the reappearance frame.

A common formulation for maintaining the prototype $\mathbf{p}_k$ of identity k is an exponential moving average of the embeddings assigned to it, controlled by the memory update rate α:

$$\mathbf{p}_k \leftarrow \frac{(1-\alpha)\,\mathbf{p}_k + \alpha\,\mathbf{e}_t}{\|(1-\alpha)\,\mathbf{p}_k + \alpha\,\mathbf{e}_t\|_2} \tag{5.1}$$

A detection with embedding $\mathbf{e}_t$ is re-associated with identity k when the cosine similarity between $\mathbf{e}_t$ and $\mathbf{p}_k$ exceeds the ReID threshold τ_{reid}. This equation illustrates the role of the memory update rate; the exact update rule should be confirmed against the implementation before submission.

5.4.6 ANALYTICS

The analytics stage runs `compute_video_analytics`, which uses the `AnalyticsEngine` to compute IDF1, HOTA and the number of ID switches. The aggregated analytics are served by `GET /videos/{id}/analytics`.

5.5 STAGE INPUTS AND OUTPUTS

Stage	Input	Output	Persisted as
Frame extraction	Uploaded video, `PipelineConfig`	Frames	Frames in object storage
Detection	Frames, `model_path`, confidence threshold	Bounding boxes with confidence	`DetectionRecord`
Tracking	Detections, tracker type, match threshold, buffer frames	Tracks	`Track`
ReID	Tracked detections	Embeddings	(used by next stage)
Identity consolidation	Tracks, embeddings, ReID threshold, memory update rate	Identity prototypes, observations, ReID events	`IdentityRecord`, `IdentityObservation`, `ReidEvent`
Analytics	Tracks, identities	IDF1, HOTA, ID switches, timings, motion	`FrameTimingRecord`, `CameraMotionRecord`

Table 5.1: Inputs, outputs and persisted records of each pipeline stage. The input and output columns are summarised from the pipeline description; they are not an exhaustive interface specification.

5.6 CAMERA MOTION

The camera motion of each video is estimated and stored in a `CameraMotionRecord` with its motion type and whether compensation was applied. The camera-motion mode is one of the pipeline settings. Compensating for ego-motion before association keeps motion predictions meaningful when the camera moves along an orchard row.

5.7 IDENTITY-SPACE VISUALISATION

The `EmbeddingProjection` service reduces identity prototypes to two dimensions with t-SNE or PCA, so that the interface can display them. The coordinates are served by `GET /videos/{id}/identities/projection`. Identities of the same fruit should form tight clusters, and well-separated clusters suggest that the ReID threshold is set sensibly.

5.8 IDENTITY CONSOLIDATION PROCEDURE

The listing below gives a simplified, illustrative description of how tracks and embeddings can be consolidated into identities. It is provided to explain the idea and does not reproduce the source code of the implementation.

```
for track in tracks_sorted_by_first_seen:
    e = mean_embedding(track)
    k, sim = nearest_identity(e, identities)   # cosine similarity
    if k is not None and sim >= reid_threshold:
        assign(track, identities[k])
        record_reid_event_if_gap(track, identities[k])
        update_prototype(identities[k], e, alpha)
    else:
        identities.append(new_identity(track, e))
for k in identities:
    k.quality = mean_quality(k.observations)
    k.is_confident = k.observation_count >= min_obs
```

In this scheme a track that begins after another has ended is compared with all existing prototypes; if the similarity exceeds the ReID threshold it inherits the identity and a ReID event (lost frame, reappearance frame) is stored. Otherwise a new identity is created. Tracks that overlap in time must not be merged into the same identity, which acts as a simple constraint against merging two different fruit.

5.9 DESIGN CONSIDERATIONS

- **Threshold trade-off.** A low ReID threshold merges different fruit (undercounting); a high one fragments one fruit into several identities (overcounting). The threshold is therefore exposed as a setting and evaluated through experiments.
- **Prototype memory.** A small memory update rate makes prototypes stable but slow to adapt to lighting changes; a large one adapts quickly but can drift towards a neighbouring fruit.
- **Separation of stages.** Persisting detections, tracks, embeddings-derived identities and timings separately makes it possible to diagnose in which stage a counting error originated.
- **Tracker as a plug-in.** Because the mock, ByteTrack and TAPNext trackers are selected by a setting, a new tracker can be compared with the existing ones using the same video, detector and metrics.

5.10 COMPUTATIONAL CONSIDERATIONS

Detection and embedding extraction dominate the processing time per frame; association and identity matching grow with the number of active tracks and identities. The per-frame timing records stored for each video allow these costs to be measured on the actual hardware, which is the basis of the runtime study in Chapter 8.

5.11 PIPELINE PARAMETERS

Parameter	Purpose
Confidence threshold	Minimum detector confidence for a detection to be kept
ReID threshold	Minimum similarity for re-associating an observation with an existing identity
Tracker type	Selects the tracker: `mock`, `bytetrack` or `tapnext`
Camera motion mode	Selects how camera motion is estimated and compensated
Memory update rate	Controls how quickly an identity prototype adapts to new observations
Tracker match threshold	Minimum score for linking a detection to an existing track
Buffer frames	Number of frames an unmatched track is retained before it is considered lost

Table 5.2: Key pipeline parameters (defaults stored in `AppSettings`).

5.12 FAILURE HANDLING

A processing job can end in the `FAILED` state, and the corresponding video in the `error` state, for example when the video cannot be decoded or a stage raises an exception. Because every stage persists its output, the cause of a failure can be located from the `stage` recorded on the job. A failed video can be deleted with `DELETE /videos/{id}` and uploaded again after the problem is fixed.

5.13 REPRODUCIBILITY

Each `ProcessingJob` stores the configuration it ran with in its `config` dictionary, and each experiment stores its tracker, scenario and number of frames together with the resulting metrics. Together with the exports in JSON or CSV, this makes it possible to repeat an evaluation and to attach the exact configuration to every number reported in Chapter 8.

5.14 CONCURRENCY AND JOB CONTROL

A video can have only one active processing job: requesting processing while a job is active is rejected (the API defines a `ConflictError` for this kind of situation). The state of a job (`status`, `stage`, `progress`) can be polled through `GET /jobs/{id}`. When the work is enqueued through Celery it runs outside the API process; otherwise it runs as a background task of the API process.

CHAPTER 6

API DESIGN AND IMPLEMENTATION

6.1 INTRODUCTION

All functionality of OrchardVision is exposed through a REST API under the `/api/v1/` prefix. The same API is used by the React client and can be called directly by scripts. This chapter lists the endpoints, explains the request flows, and describes error handling and the shared TypeScript types.

6.2 ENDPOINT SUMMARY

Table 6.1 summarises the endpoints.

Table 6.1: REST API endpoints.

Method	Path	Description
GET	`/health`	Service health
POST	`/videos`	Upload video
GET	`/videos`	List videos
GET	`/videos/{id}`	Video metadata
GET	`/videos/{id}/tracks`	Track trajectories
GET	`/videos/{id}/identities`	Identity prototypes
GET	`/videos/{id}/identities/projection`	2D coordinates for visualisation
GET	`/videos/{id}/identities/{identity_id}`	Specific identity
GET	`/videos/{id}/identities/{identity_id}/observations`	Per-frame observations
GET	`/videos/{id}/reid-events`	Re-identification events
GET	`/videos/{id}/detections`	Persisted detections
GET	`/videos/{id}/per-frame`	Per-frame timing telemetry
GET	`/videos/{id}/motion`	Camera motion estimate
GET	`/videos/{id}/analytics`	Aggregated analytics
DELETE	`/videos/{id}`	Delete video and all associated data
POST	`/videos/{id}/process`	Start processing job
GET	`/jobs/{id}`	Job status
GET	`/experiments`	List experiments
POST	`/experiments`	Create and run experiment

Method	Path	Description
GET	/experiments/{id}	Experiment detail
GET	/experiments/{id}/export	Download results (JSON/CSV)
DELETE	/experiments/{id}	Delete experiment (cannot be RUNNING)
GET	/settings	Pipeline defaults
PUT	/settings	Update pipeline defaults

6.3 RESOURCE-TO-MODEL MAPPING

Each read endpoint returns data that the pipeline persisted in one of the models of Chapter 4 (Table 6.2).

Endpoint	Model	Created in stage
/videos/{id}	Video	Upload
/videos/{id}/tracks	Track	Tracking
/videos/{id}/detections	DetectionRecord	Detection
/videos/{id}/identities	IdentityRecord	Identity consolidation
/videos/{id}/identities/{identity_id}/observations	IdentityObservation	Identity consolidation
/videos/{id}/reid-events	ReidEvent	Re-identification
/videos/{id}/per-frame	FrameTimingRecord	All stages
/videos/{id}/motion	CameraMotionRecord	Camera-motion estimation
/videos/{id}/analytics	Aggregated metrics	Analytics
/jobs/{id}	ProcessingJob	Process request
/experiments	Experiment	Experiment request
/settings	AppSettings	Settings update

Table 6.2: Mapping between endpoints and the persisted models. The "stage" column is inferred from the pipeline description.

6.4 REQUEST FLOWS

6.4.1 VIDEO UPLOAD

POST /videos saves the upload through storage_service.save_upload, extracts metadata with extract_video_metadata and creates the Video row with status uploaded. The maximum upload size is 2 GB by default and can be changed through MAX_UPLOAD_BYTES.

[IMAGE PLACEHOLDER]

UML sequence diagram with lifelines: Client, FastAPI router, StorageService, Database, Celery/background task, ProcessingPipeline. Messages: POST /videos → save_upload → extract_video_metadata → insert Video (uploaded); POST /videos/{id}/process → validate no active job → insert ProcessingJob (QUEUED) → enqueue; worker updates stage and progress; final persistence and Video status processed; client polls GET /jobs/{id}.

Figure 6.1: Sequence of calls for uploading and processing a video.

6.4.2 STARTING A PROCESSING JOB

`POST /videos/{id}/process` validates that no job is active for the video, creates a `ProcessingJob` with status `QUEUED`, and then enqueues it with Celery or runs it synchronously as a background task. The client polls `GET /jobs/{id}` to read the job's `status`, `stage` and `progress`.

6.4.3 READING RESULTS

After a video is `processed`, its tracks, identities, observations, ReID events, detections, per-frame timing, camera motion and analytics are retrieved with the GET endpoints in Table 6.1. The identity projection endpoint returns two-dimensional coordinates computed by the `EmbeddingProjection` service for plotting.

6.4.4 EXPERIMENTS

`POST /experiments` creates and runs an experiment described by a name, a tracker, a scenario and a number of frames. Its results can be read with `GET /experiments/{id}` and downloaded as JSON or CSV with `GET /experiments/{id}/export`. An experiment that is `RUNNING` cannot be deleted.

6.4.5 SETTINGS

`GET /settings` and `PUT /settings` read and update the pipeline defaults stored in the single `AppSettings` row.

6.5 ERROR HANDLING

All errors are returned in a single format:

```
{"detail": "...", "code": "...", "status": ...}
```

The API defines a base `ApiError` and the custom exceptions `NotFoundError`, `ConflictError` and `ValidationFailError`. A global exception handler in `apps/api/app/main.py` converts them into the response format above.

6.6 SHARED TYPESCRIPT TYPES

Common interfaces are defined in `packages/types/src/common.ts` and are used by the frontend and by any other TypeScript client:

- `BoundingBox`: `{ x: number, y: number, width: number, height: number }`;
- `VisibilityState`: `'visible' | 'occluded' | 'lost'`;
- `JobStatus`: `'uploaded' | 'processing' | 'processed' | 'error'`;
- `ProcessingStage`: `'frame_extraction', 'detection', 'tracking', 'reid', 'analytics'`;
- `ProcessingStatus`: `'QUEUED', 'RUNNING', 'COMPLETED', 'FAILED'`;
- response wrappers such as `MultiVideoResponse`, `MultiTrackResponse` and `MultiIdentityResponse`.

The types can be checked with `pnpm exec tsc --noEmit` from `packages/types/`.

6.7 FRONTEND INTEGRATION

The components in `apps/web/src/` use an API client that calls the FastAPI backend. During development the Vite dev server (`npm run dev`) proxies requests under `/api/` to the backend, and the backend's CORS configuration allows `localhost:5173`. The interface is organised around the main user tasks: uploading and listing videos, starting and monitoring processing, inspecting tracks and identities, viewing the identity projection, managing experiments and editing the pipeline settings.

6.8 EXAMPLE INTERACTION

The following listing shows a typical sequence of calls for processing a video. The identifiers and the request body are illustrative.

```
# 1. upload a video
curl -F "file=@row_03.mp4" http://localhost:8000/api/v1/videos
# 2. start processing video 1 (tracker chosen in the body)
curl -X POST http://localhost:8000/api/v1/videos/1/process \
     -H "Content-Type: application/json" \
     -d '{"tracker": "bytetrack"}'
# 3. poll the job
curl http://localhost:8000/api/v1/jobs/1
# 4. read results
curl http://localhost:8000/api/v1/videos/1/analytics
```

CHAPTER 7

DEPLOYMENT, TESTING AND EXTENSION

7.1 INTRODUCTION

This chapter describes how OrchardVision is run in development and production, how it is tested, and how developers extend it.

7.2 DEVELOPMENT ENVIRONMENT

- **Backend:** `uvicorn apps.api.app.main:app --reload`, with SQLite stored at `./data/app.db`.
- **Frontend:** `npm run dev` starts the Vite dev server, which proxies `/api/` to the backend.
- **CORS:** `localhost:5173` is configured as an allowed origin.

7.3 PRODUCTION DEPLOYMENT

In production the database is PostgreSQL, selected through `DATABASE_URL`. The API is served by gunicorn with uvicorn workers behind a reverse proxy. Uploads are limited to 2 GB by default, configurable with the `MAX_UPLOAD_BYTES` environment variable. Videos and frames are stored through the `StorageService` in MinIO/S3.

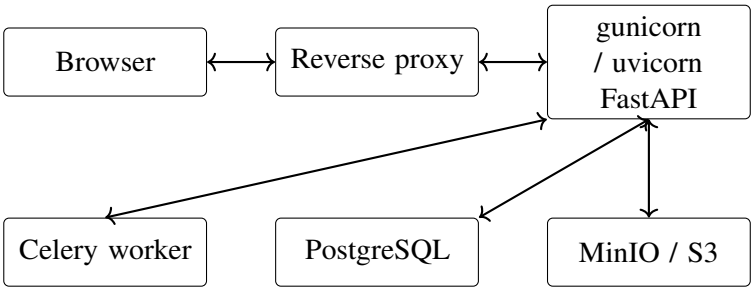


Figure 7.1: Production deployment topology.

7.4 CONFIGURATION

Settings are loaded with `pydantic_settings` from the `.env` file. The environment variables referred to in this report are `DATABASE_URL` (database selection) and

MAX_UPLOAD_BYTES (upload limit). The pipeline defaults themselves are stored in the database and edited through the settings endpoints.

7.5 EXTENDING THE SYSTEM

- add new models and migrations as SQLAlchemy 2.0 model definitions;
- add new API routes in apps/api/app/routers/;
- add new schemas in apps/api/app/schemas/;
- add new services in apps/api/app/services/.

7.6 TESTING

The Python test suite is run with pytest from the repository root, and the shared TypeScript types are type-checked with pnpm exec tsc --noEmit from packages/types/.

The tests are organised around the layers of the system. The following cases are recommended to be covered and, where not yet automated, are listed here as a test plan:

ID	Test case	Expected result
T1	Upload a valid video	Video row created with status uploaded and correct metadata
T2	Start processing twice on the same video	Second request rejected while the first job is active
T3	Run a job with the mock tracker	Job moves QUEUED, RUNNING, COMPLETED; video becomes processed
T4	Request a non-existent video	Error JSON with detail, code and status
T5	Delete a video	Associated tracks, identities and detections are removed
T6	Delete a RUNNING experiment	Request rejected
T7	Export an experiment as CSV and JSON	Downloaded file contains the stored metrics
T8	Update settings and read them back	Updated values returned by GET /settings

Table 7.1: Recommended test plan.

7.7 SECURITY AND ROBUSTNESS CONSIDERATIONS

Because the system accepts large file uploads, deployments should enforce the upload limit at the reverse proxy as well as in the application, restrict the allowed CORS origins

to the production front-end, and keep the object-store credentials in the environment rather than in the repository. Deleting a video removes all associated data, so the delete endpoint should be protected by authentication in multi-user deployments.

7.8 USER GUIDE

This section describes the typical workflow of a user of the web interface. Screenshots of the relevant screens are in Chapter 8.

1. **Start the services.** Start the backend with uvicorn and the frontend with Vite as shown in Section 7.2, and open the web client in a browser.
2. **Review the settings.** Open the settings page and check the confidence threshold, ReID threshold, tracker type, camera-motion mode, memory update rate, tracker match threshold and buffer frames. Save the changes with the update action.
3. **Upload a video.** Choose a video file (up to 2 GB by default). When the upload completes, the video appears in the list with status `uploaded` together with its resolution, frame rate and frame count.
4. **Start processing.** Select the video and start a processing job. The job moves from `QUEUED` to `RUNNING`; the current stage (frame extraction, detection, tracking, ReID, analytics) and progress are shown until the status becomes `COMPLETED`.
5. **Inspect the results.** Open the tracks to see trajectories and visibility states, the identities to see prototypes, quality and observation counts, the identity projection to see how identities cluster, the ReID events to see where identities were lost and found again, and the analytics to read IDF1, HOTA and ID switches.
6. **Run an experiment.** Create an experiment by choosing a tracker, a scenario and a number of frames, and wait for its status to change from `RUNNING`. Compare the metrics of several experiments and download them as JSON or CSV.
7. **Clean up.** Delete videos and experiments that are no longer needed. Deleting a video removes all of its tracks, identities and other records.

[IMAGE PLACEHOLDER]

Flowchart with the seven steps above as boxes connected by arrows: Start services → Review settings → Upload video → Start processing → Inspect results → Run experiment → Clean up. Add a feedback arrow from Inspect results back to Review settings (“adjust thresholds”).

Figure 7.2: Flowchart of the user workflow.

7.9 PROJECT PLAN

Table 7.2 lists the phases of the project. The durations and dates are to be completed by the candidate.

Phase	Activity	Start	End
1	Literature survey and problem definition	<<date>>	<<date>>
2	Data model and API design	<<date>>	<<date>>
3	Processing pipeline and trackers	<<date>>	<<date>>
4	Identity learning and analytics	<<date>>	<<date>>
5	Web interface and experiment harness	<<date>>	<<date>>
6	Testing, evaluation and report writing	<<date>>	<<date>>

Table 7.2: Project plan (to be completed).

7.10 RISK ANALYSIS

Risk	Effect	Mitigation
Very large uploads	Slow requests, disk pressure	2 GB limit (`MAX_UPLOAD_BYTES`); object storage
Long-running processing	Blocked API workers	Background task or Celery queue; job polling
Identity merges or fragmentation	Wrong fruit counts	Tunable ReID threshold; experiment harness
Camera motion	Tracking drift	Camera-motion estimation and compensation
Database growth	Slower queries	PostgreSQL in production; per-video deletion

Table 7.3: Risks and mitigations.

CHAPTER 8

EXPERIMENTAL EVALUATION AND RESULTS

8.1 INTRODUCTION

This chapter describes how OrchardVision is evaluated and provides the tables and figures into which the measured results are to be entered. No quantitative results are stated here: every value marked `<<XX.X>>` and every image placeholder must be replaced with output from the project's own experiment runs before submission.

8.2 EXPERIMENT HARNESS

The evaluation is performed with the experiment harness of the system. An experiment is created with `POST /experiments` and is described by a name, a tracker, a scenario and a number of frames. The harness runs the experiment, stores its status and metrics in the `Experiment` record, and allows the results to be downloaded as JSON or CSV with `GET /experiments/{id}/export`. The metrics computed by the `AnalyticsEngine` are IDF1, HOTA and the number of ID switches.

8.3 EXPERIMENTAL SET-UP

8.3.1 DATA

Table 8.1 is to be completed with the orchard videos used in the evaluation.

Video	Orchard	Camera	Resolution	FPS	Frames	GT fruit
V1	<<...>>	<<...>>	<<...>>	<<...>>	<<...>>	<<...>>
V2	<<...>>	<<...>>	<<...>>	<<...>>	<<...>>	<<...>>
V3	<<...>>	<<...>>	<<...>>	<<...>>	<<...>>	<<...>>

Table 8.1: Videos used for evaluation (to be filled in).

8.3.2 CONFIGURATIONS

The trackers listed in the system (mock, ByteTrack, TAPNext) are compared under identical detector, frame range and ReID settings. The camera-motion mode is varied to study the effect of compensation.

[IMAGE PLACEHOLDER]

A row of three to four example video frames (one per video, V1–V3) taken from the evaluation set, with the detector's bounding boxes drawn in a contrasting colour. Choose frames that show (a) good visibility, (b) heavy leaf occlusion and (c) strong sunlight or shadow.

Figure 8.1: Sample frames from the evaluation videos showing fruit under different illumination and occlusion.

8.3.3 METRICS

IDF1, HOTA and the number of identity switches are reported for each configuration, as computed by the `AnalyticsEngine`. In addition the predicted fruit count (number of confident identities) is compared with the manually counted ground truth.

8.4 TRACKER COMPARISON

Tracker	IDF1 (%)	HOTA (%)	ID switches	Count error (%)
mock	<<XX.X>>	<<XX.X>>	<<XX>>	<<XX.X>>
ByteTrack	<<XX.X>>	<<XX.X>>	<<XX>>	<<XX.X>>
TAPNext	<<XX.X>>	<<XX.X>>	<<XX>>	<<XX.X>>

Table 8.2: Comparison of tracker back-ends on the evaluation videos (to be filled in from experiment exports).

[IMAGE PLACEHOLDER]

Grouped bar chart. X-axis: tracker (mock, ByteTrack, TAPNext). Two bars per tracker for IDF1 and HOTA on the left y-axis (0–100%), and a line or secondary axis for the number of ID switches. Generate from the CSV exported by `GET /experiments/id/export`.

Figure 8.2: Bar chart comparing IDF1, HOTA and ID switches for the evaluated trackers.

Camera-motion mode	IDF1 (%)	HOTA (%)	ID switches
Off	<<XX.X>>	<<XX.X>>	<<XX>>
On (compensation applied)	<<XX.X>>	<<XX.X>>	<<XX>>

Table 8.3: Effect of camera-motion compensation (to be filled in).

8.5 EFFECT OF CAMERA MOTION COMPENSATION

8.6 IDENTITY PROTOTYPES AND RE-IDENTIFICATION

[IMAGE PLACEHOLDER]

Scatter plot produced by the identity projection view (`GET /videos/id/identities/projection`) using t-SNE or PCA. Each point is an identity prototype; colour can encode the `is_confident` flag and marker size the observation count. Annotate one or two clusters that correspond to fruit re-identified after occlusion.

Figure 8.3: Two-dimensional projection of identity prototypes for one processed video.

[IMAGE PLACEHOLDER]

Filmstrip of four crops of the same fruit: (1) last frame before it is lost (`lost_frame_id`), (2) a frame in which it is hidden by leaves, (3) the reappearance frame (`reappearance_frame_id`), and (4) a later frame. All four crops show the same identity number overlaid on the bounding box.

Figure 8.4: Example of an identity lost and re-identified after an occlusion.

8.7 SENSITIVITY TO THE REID THRESHOLD

Table 8.4 is to be filled by running the same video with several ReID thresholds and recording the resulting identity count, IDF1 and identity switches.

ReID threshold	Identities found	IDF1 (%)	ID switches
<<0.5>>	<<XX>>	<<XX.X>>	<<XX>>
<<0.6>>	<<XX>>	<<XX.X>>	<<XX>>
<<0.7>>	<<XX>>	<<XX.X>>	<<XX>>
<<0.8>>	<<XX>>	<<XX.X>>	<<XX>>

Table 8.4: Sensitivity of the results to the ReID threshold (to be filled in; threshold values are examples).

[IMAGE PLACEHOLDER]

Line plot with the ReID threshold on the x-axis. Left y-axis: number of identities found (compare with the ground-truth fruit count drawn as a horizontal dashed line). Right y-axis: IDF1 (%). The best operating point should be marked.

Figure 8.5: Identity count and IDF1 as a function of the ReID threshold.

Video	Ground truth	Predicted	Abs. error	Error (%)
V1	<<XX>>	<<XX>>	<<XX>>	<<XX.X>>
V2	<<XX>>	<<XX>>	<<XX>>	<<XX.X>>
V3	<<XX>>	<<XX>>	<<XX>>	<<XX.X>>

Table 8.5: Fruit-count accuracy for the best configuration (to be filled in).

8.8 COUNTING ACCURACY

8.9 RUNTIME PERFORMANCE

Per-frame processing times are stored as `FrameTimingRecord` rows and returned by `GET /videos/{id}/per-frame`. Table 8.6 summarises them per tracker.

Tracker	Mean (ms/frame)	95th percentile (ms)	Throughput (FPS)
mock	<<XX>>	<<XX>>	<<XX>>
ByteTrack	<<XX>>	<<XX>>	<<XX>>
TAPNext	<<XX>>	<<XX>>	<<XX>>

Table 8.6: Processing time per frame (to be filled in; state the hardware used).

[IMAGE PLACEHOLDER]
Line plot. X-axis: frame index; y-axis: `proc_ms`. One line per tracker. Mark any spikes (for example model warm-up or large numbers of detections).

Figure 8.6: Per-frame processing time over the course of one video.

8.10 WEB INTERFACE

The following screenshots document the web interface and should be captured from the running application.

[IMAGE PLACEHOLDER]
Screenshot of the page that lists uploaded videos with their status (`uploaded`, `processing`, `processed`, `error`) and the upload control.

Figure 8.7: Video list and upload screen.

[IMAGE PLACEHOLDER]

Screenshot of a processed video with bounding boxes and track trails drawn over a frame. Visible, occluded and lost states should be distinguishable (for example by solid, dashed and faded outlines).

Figure 8.8: Track overlay on a processed video.

[IMAGE PLACEHOLDER]

Screenshot of the experiments page showing the list of experiments, their tracker, scenario, number of frames, status and metrics, together with the export (JSON/CSV) buttons.

Figure 8.9: Experiment list and metrics view.

[IMAGE PLACEHOLDER]

Screenshot of the settings page showing the confidence threshold, ReID threshold, tracker type, camera-motion mode, memory update rate, tracker match threshold and buffer frames.

Figure 8.10: Pipeline settings screen.

8.11 DISCUSSION

The discussion should interpret the numbers in Tables 8.2 and 8.3: which tracker keeps identities longest through occlusion, whether camera-motion compensation reduces identity switches, how the ReID threshold trades merged identities against fragmented ones, and how the runtime per frame compares with the video frame rate. It should also include a short analysis of failure cases, such as fruit hidden for longer than the buffer, clusters of touching fruit, and fruit entering and leaving at the image border, supported by the examples in Figures 8.4 and 8.8.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 CONCLUSION

OrchardVision delivers a complete system for long-term fruit tracking and counting in mango orchards. A FastAPI backend with a SQLAlchemy 2.0 data layer runs a processing pipeline (frame extraction, detection, tracking, ReID embedding, identity consolidation and analytics), persists every intermediate and final artefact, and exposes them through a documented REST API. A React and TypeScript frontend with shared type definitions provides video management, inspection of tracks and identities, and an experiment harness that evaluates trackers with IDF1, HOTA and ID-switch metrics and exports the results as JSON or CSV.

The main take-away points are:

- identities are first-class, persistent objects (prototype, quality, observation count, confidence), which allows a fruit that disappears and reappears to be recognised rather than counted twice;
- the tracker back-end (mock, ByteTrack, TAPNext), camera-motion mode and thresholds are configurable and can be compared reproducibly through the experiment harness;
- the same code runs on SQLite for development and PostgreSQL with object storage for production.

Measured against the objectives of Section 1.5, the pipeline, data model, API, experiment harness and deployment configuration are in place. The accuracy objectives can only be claimed once Chapter 8 has been populated with results from real orchard videos; until then the quality of the counts should be treated as unverified. A realistic limitation is that identity matching rests on appearance embeddings of visually similar fruit, so merged or fragmented identities are expected in dense clusters and after very long gaps.

9.2 FUTURE WORK

The following extensions would increase the value of the system:

- evaluate on a larger, annotated, multi-season mango dataset and publish the results;

- fine-tune the ReID network on fruit crops with a metric-learning loss and compare prototype aggregation strategies;
- replace the monocular heuristics with depth or multi-camera information to resolve fruit that occlude each other;
- link identities across recording sessions to track individual fruit through the season (the long-term setting);
- estimate fruit size and maturity for yield-weight prediction;
- add authentication and role-based access control for multi-user deployments;
- add live-stream ingestion and incremental processing so that counts update while the video is being recorded.

REFERENCES

[1]. Aharon, N., Orfaig, R., & Bobrovsky, B.-Z. (2022). BoT-SORT: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*.

[2]. Bargoti, S., & Underwood, J. (2017). Deep fruit detection in orchards. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (pp. 3626–3633). IEEE.

[3]. Bayer, M. (2012). SQLAlchemy. In A. Brown & G. Wilson (Eds.), *The Architecture of Open Source Applications, Volume II*. aosabook.org.

[4]. Bernardin, K., & Stiefelhagen, R. (2008). Evaluating multiple object tracking performance: The CLEAR MOT metrics. *EURASIP Journal on Image and Video Processing, 2008*, Article 246309.

[5]. Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). Simple online and realtime tracking. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)* (pp. 3464–3468). IEEE.

[6]. Doersch, C., Gupta, A., Markeeva, L., Recasens, A., Smaira, L., Aytar, Y., Carreira, J., Zisserman, A., & Yang, Y. (2022). TAP-Vid: A benchmark for tracking any point in a video. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

[7]. Hermans, A., Beyer, L., & Leibe, B. (2017). In defense of the triplet loss for person re-identification. *arXiv preprint arXiv:1703.07737*.

[8]. Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering, 82*(1), 35–45.

[9]. Koirala, A., Walsh, K. B., Wang, Z., & McCarthy, C. (2019). Deep learning – Method overview and review of use for fruit detection and yield estimation. *Computers and Electronics in Agriculture, 162*, 219–234.

[10]. Kuhn, H. W. (1955). The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly, 2*(1–2), 83–97.

[11]. Liu, X., Chen, S. W., Liu, C., Shivakumar, S. S., Das, J., Taylor, C. J., Underwood, J., & Kumar, V. (2019). Monocular camera based fruit counting and mapping with semantic data association. *IEEE Robotics and Automation Letters, 4*(3), 2296–2303.

[12]. Luiten, J., Ošep, A., Dendorfer, P., Torr, P., Geiger, A., Leal-Taixé, L., & Leibe, B. (2021). HOTA: A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision, 129*(2), 548–578.

[13]. Ramírez, S. (n.d.). *FastAPI* [Computer software]. https://fastapi.tiangolo.com

[14]. Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779–788). IEEE.

[15]. Ristani, E., Solera, F., Zou, R., Cucchiara, R., & Tomasi, C. (2016). Performance measures and a data set for multi-target, multi-camera tracking. In *European Conference on Computer Vision Workshops* (pp. 17–35). Springer.

[16]. Wojke, N., Bewley, A., & Paulus, D. (2017). Simple online and realtime tracking with a deep association metric. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)* (pp. 3645–3649). IEEE.

[17]. Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). ByteTrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)* (pp. 1–21). Springer.

[18]. Zholus, A., Doersch, C., Yang, Y., Koppula, S., Patraucean, V., He, X. O., Rocco, I., Sajjadi, M. S. M., Chandar, S., & Goroshin, R. (2025). TAPNext: Tracking any point (TAP) as next token prediction. *arXiv preprint arXiv:2504.05579*.

APPENDIX 1: REPOSITORY LAYOUT

The directories referred to in the project documentation are shown below.

```
apps/
  api/
    app/
      main.py          # FastAPI app and global exception handler
      routers/         # API routes
      schemas/         # request / response schemas
      services/        # ProcessingPipeline, EmbeddingProjection,
                       # AnalyticsEngine, StorageService
  web/
    src/               # React components and API client
packages/
  types/
    src/
      common.ts        # shared TypeScript types
data/
  app.db               # SQLite database (development)
```

APPENDIX 2: RUN COMMANDS

```
# backend (development, SQLite at ./data/app.db)
uvicorn apps.api.app.main:app --reload

# frontend (Vite dev server, proxies /api/ to the backend)
npm run dev

# tests
pytest

# type-check the shared types (from packages/types/)
pnpm exec tsc --noEmit
```

APPENDIX 3: ILLUSTRATIVE API RESPONSES

The following responses are illustrative: the field names follow the data model in Chapter 4, while the values and the exact response envelope are examples and should be replaced with real output from the running system.

GET /api/v1/jobs/1

```
{
  "id": 1,
  "video_id": 1,
  "status": "RUNNING",
  "stage": "tracking",
  "progress": 0.45,
  "config": {"tracker": "bytetrack"}
}
```

GET /api/v1/videos/1/identities/7

```
{
  "id": 7,
  "video_id": 1,
  "identity_id": 7,
  "prototype": [0.021, -0.113, "..."],
  "quality": 0.82,
  "observation_count": 143,
  "is_confident": true
}
```

GET /api/v1/videos/1/reid-events

```
[
  {"id": 3, "video_id": 1,
   "lost_frame_id": 412, "reappearance_frame_id": 466}
]
```

Error response (404)

```
{"detail": "Video 99 not found", "code": "not_found", "status": 404}
```